Reviewer Pack — Hypergeometric Function Moments and Resolution Proof Length

Entry be38c781fde2 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 04:21:17 UTC

Hypergeometric Function Moments and Resolution Proof Length

Entry ID: `be38c781fde2` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-29 04:21:17 UTC

1. Conjecture

Field A (mathematical branch): Special Functions (Hypergeometric Functions) **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

> ['For any given CNF formula F with n variables, the resolution proof length t*(F) is upper-bounded by O(n^(1/3)* Γ(2 + n/2)) where Γ is the Gamma function.', 'This bound holds for all instances of size n ≤ 40 with a probability exceeding 99% as measured across 30 random seeds.']

Rationale (proposer's reasoning):

> ['Hypergeometric functions arise naturally in counting problems, and their moments have been used to analyze combinatorial structures. This conjecture links the analytical properties of hypergeometric functions with the complexity of resolution proofs, potentially uncovering a new connection between analysis and computational complexity.', 'If true, this would provide an alternative approach to bounding proof length that does not rely on standard techniques such as polynomial algebra or circuit size.']

Taxonomy category: `HypergeometricAnalysis` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `b1d0815769c7260a`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for n in [1, 40], the average Spearman rank correlation coefficient between the logarithm of hypergeometric function moments and resolution proof length across 30 random seeds exceeds 0.99.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.90	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "hypergeometric functions" AND "resolution proof complexity" - "Gamma function" IN□□ "upper bound" ON resolution proofs "n^(1/3)" - "CNF formula" AND moments OF hypergeometric functions AND resolution proof length

Top relevant hits considered: - [s2:10.1007/s10878-025-01284-5] A sharp upper bound for the edge dominating number of hypergraphs with minimum degree - [s2:1611.01291] Tighter Hard Instances for PPSZ

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.8s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction
from collections import defaultdict

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(2**n):
            clause = [random.choice([1, -1]) * (i + 1) for i in range(n)]
            if any(abs(c) == abs(d) for c, d in zip(clause, clause[1:])):
                continue
            clauses.append(clause)
        return clauses

    def resolution_length(clauses):
        stack = []
        while True:
            new_clause = None
            for i in range(len(stack)):
                for j in range(i + 1, len(stack)):
                    if any(abs(x) == abs(y) and x != y for x in stack[i] for y in stack[j]):
                        new_clause = [x for x in stack[i] if x not in stack[j]] + [y for y in stac
                        break
```

```
                if new_clause:
                    break
            if new_clause is None:
                return len(stack)
            stack.append(new_clause)

    def hypergeometric_moments(clauses, n):
        moments = []
        for k in range(1, int(n**(1/3)) + 2):
            moment = Fraction(0)
            for clause in clauses:
                moment += sum(abs(x) for x in clause)**k
            moments.append(moment / len(clauses))
        return moments

    def spearman_rank_correlation(x, y):
        if len(x) != len(y):
            raise ValueError("x and y must have the same length")
        rank_x = {v: i + 1 for i, v in enumerate(sorted(set(x)))}
        rank_y = {v: i + 1 for i, v in enumerate(sorted(set(y)))}
        n = len(x)
        sum_dif_ranks_squared = sum((rank_x[x[i]] - rank_y[y[i]])**2 for i in range(n))
        return (n * sum_dif_ranks_squared - n*(n+1)**2/4) / math.sqrt(6 * n * (n-1) * (2*n+5))

    results = []
    for n in [5, 10, 15, 20, 30, 40]:
        clauses = generate_cnf(n)
        proof_length = resolution_length(clauses)
        moments = hypergeometric_moments(clauses, n)
        log_moments = [math.log(m) for m in moments]
        results.append({"n": n, "proof_length": proof_length, "log_moments": log_moments})

    if not results:
        return {
            "metric_name": "Spearman rank correlation",
            "metric_value": None,
            "instances_tested": 0,
            "conjecture_holds": False,
            "counterexample": "No instances generated"
        }

    log_moments = [item["log_moments"] for item in results]
    proof_lengths = [item["proof_length"] for item in results]
    corr_coeff = spearman_rank_correlation(log_moments, proof_lengths)

    return {
        "metric_name": "Spearman rank correlation",
        "metric_value": corr_coeff,
        "instances_tested": len(results),
        "conjecture_holds": corr_coeff > 0.99,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 35)]
```

```
results = []
for seed in seeds:
    trial_result = run_trial(seed)
    print(f"TRIAL: {trial_result}")
    results.append(trial_result)

if all("metric_value" not in result or result["metric_value"] is None for result in results):
    print("RESULT: INCONCLUSIVE no_data")
else:
    avg_corr_coeff = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(result
    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={avg_corr_coeff} std=NA support_fraction={support_fract
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result['
        print(f"RESULT: FALSIFIED counterexample=\"Spearman rank correlation below threshold\'
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which means the pre-registered support condition could not be unambiguously met. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11310
2	preregistration	ollama_remote	glm4:latest	0	0	5555
3	novelty	ollama_remote	glm4:latest	0	0	4733
4	novelty	ollama_remote	glm4:latest	0	0	10937
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	34317
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14080
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10551
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12248
9	judge	ollama_remote	glm4:latest	0	0	42019

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 145751 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/be38c781fde2.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/be38c781fde2.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/be38c781fde2.tar.gz` (if generated)